INDEX

SR. NO	DATE	TITLE	PAGE.NO	SIGN
1		Implement the Breadth First Search algorithm to solve a given problem.		
2		Implement the Iterative Depth First Search algorithm to solve the same problem.		
3		Implement the A* Search algorithm for solving a pathfinding problem.		
4		Implement the Recursive Best-First Search algorithm for the same problem.		
5		Implement Decision-Tree learning algorithm.		
6		Implement Naive-Bayes learning algo for RWP (Restaurant Waiting Problem).		
7		Implement feed forward back propagation neural network learning algorithm.		
8		Implement AdaBoost (Adaptive Boosting) learning algorithm		
9		Implement Reinforcement Learning algorithm.		

Common File:- RMP.py

```
dict_hn={'Arad':336,'Bucharest':0,'Craiova':160,'Drobeta':242,'Eforie':161,
    'Fagaras':176,'Giurgiu':77,'Hirsova':151,'Iasi':226,'Lugoj':244,
    'Mehadia':241,'Neamt':234,'Oradea':380,'Pitesti':100,'Rimnicu':193,
    'Sibiu':253,'Timisoara':329,'Uzriceni':80,'Vaslui':199,'Zerind':374}
dict_gn=dict(
  Arad=dict(Zerind=75,Timisoara=118,Sibiu=
  140),
  Bucharest=dict(Uriceni=85,Giurgiu=90,Pitesti=101,Fag
  aras=211),
  Craiova=dict(Drobeta=120,Pitestti=138,Rimnicu=146),
  Drobeta=dict(Mehadia=75,Craiova=120),
  Eforie=dict(Hirsova=86),
  Fagaras=dict(Sibiu=99,Bucharest=211),
  Giurgiu=dict(Bucharest=90),
  Hirsova=dict(Eforie=86,Urziceni=98),
  Iasi=dict(Neamt=87,Vaslui=92),
  Lugoj=dict(Mehadia=70,Timisoara=111),
  Mehadia=dict(Lugoj=70,Drobeta=75),
  Neamt=dict(Iasi=87),
  Oradea=dict(Zerind=71,Sibiu=151),
  Pitesti=dict(Rimnicu=97,Bucharest=101,Craio
  va=138),
  Rimnicu=dict(Sibiu=80,Pitesti=97,Craiova=1
  46),
  Sibiu=dict(Rimnicu=80,Fagaras=99,Arad=140,Oradea=
  151), Timisoara=dict(Lugoj=111,Arad=118),
```

```
    Uzriceni=dict(Bucharest=85,Hirsova=98,Vaslui=142),

    Vaslui=dict(Iasi=92,Urziceni=142),

    Zerind=dict(Oradea=71,Arad=75)

)
```

Common File:- RWP.py

```
rwp_examples = dict(

    x1=dict(Alt='Y', Bar='N', Fri='N',Hun='Y',Pat='S',Price='$$$',Rain='N',Res='Y',Type='F',Est='0-
    10',ans='Y'),

    x2=dict(Alt='Y',  Bar='N',  Fri='N',Hun='Y',Pat='F',Price='$',Rain='N',Res='N',Type='T',Est='30-
    60',ans='N'),

    x3=dict(Alt='N', Bar='Y', Fri='N',Hun='N',Pat='S',Price='$',Rain='N',Res='N',Type='B',Est='0-
    10',ans='Y'),

    x4=dict(Alt='Y',  Bar='N',  Fri='Y',Hun='Y',Pat='F',Price='$',Rain='Y',Res='N',Type='T',Est='10-
    30',ans='Y'),

    x5=dict(Alt='Y', Bar='N',
    Fri='Y',Hun='N',Pat='F',Price='$$$',Rain='N',Res='Y',Type='F',Est='>60',ans='N'),

    x6=dict(Alt='N', Bar='Y', Fri='N',Hun='Y',Pat='S',Price='$$',Rain='Y',Res='Y',Type='I',Est='0-
    10',ans='Y'),

    x7=dict(Alt='N', Bar='Y', Fri='N',Hun='N',Pat='N',Price='$',Rain='Y',Res='N',Type='B',Est='0-
    10',ans='N'),

    x8=dict(Alt='N', Bar='N', Fri='N',Hun='Y',Pat='S',Price='$$',Rain='Y',Res='Y',Type='T',Est='0-
    10',ans='Y'),

    x9=dict(Alt='N', Bar='Y',
    Fri='Y',Hun='N',Pat='F',Price='$',Rain='Y',Res='N',Type='B',Est='>60',ans='N'),

    x10=dict(Alt='Y', Bar='Y',
Fri='Y',Hun='Y',Pat='F',Price='$$$',Rain='N',Res='Y',Type='I',Est='10- 30',ans='N'),

    x11=dict(Alt='N', Bar='N', Fri='N',Hun='N',Pat='N',Price='$',Rain='N',Res='N',Type='T',Est='0-
    10',ans='N'),

    x12=dict(Alt='Y', Bar='Y', Fri='Y',Hun='Y',Pat='F',Price='$',Rain='N',Res='N',Type='B',Est='0-
    10',ans='Y')
```

Practical No: 1

Aim:- Implement the Breadth First Search algorithm to solve a given problem.

Input:-

```
import queue as Q
from RMP import dict_gn

start='Arad'
goal='Bucharest'
result=''

def BFS(city, cityq,
   visitedq): global result
   if city==start:
      result=result+' '+city
   for eachcity in
      dict_gn[city].keys(): if
      eachcity==goal:
         result=result+'
         '+eachcity return
      if eachcity not in cityq.queue and eachcity not in visitedq.queue:
         cityq.put(eachcity)
         result=result+'
   '+eachcity
   visitedq.put(city)
   BFS(cityq.get(),cityq,visi
   tedq)

def main():
```

```
    cityq=Q.Queue()
    visitedq=Q.Queue()
    BFS(start, cityq,
    visitedq)
    print("BFS Traversal from ",start," to ",goal,"
    is: ") print(result)
main()
```

Output:-

```
======================== RESTART: C:/Users/chika/AIp1.py ========================
BFS Traversal from  Arad  to  Bucharest  is:
 Arad Zerind Timisoara Sibiu Oradea Lugoj Rimnicu Fagaras Mehadia Pitesti Craiov
a Bucharest
```

Practical No: 2

Aim:- Implement the Iterative Depth First Search algorithm to solve the same problem.

Input:-

```
import queue as Q
from RMP import dict_gn

start='Arad'
goal='Bucharest'
result=''

def DLS(city, visitedstack, startlimit,
    endlimit): global result
    found=0
    result=result+city+' '
    visitedstack.append(
    city) if city==goal:
        return 1
  if startlimit==endlimit:
        return 0
    for eachcity in
        dict_gn[city].keys(): if
        eachcity not in
        visitedstack:
            found=DLS(eachcity, visitedstack, startlimit+1,
            endlimit) if found:
                return found
def IDDFS(city, visitedstack,
    endlimit): global result
```

```
    for i in range(0, endlimit):
        print("Searching at Limit:
        ",i) found=DLS(city,
        visitedstack, 0, i) if found:
            print("Found
            ") break
        else:
            print("Not Found! "

         print(result)
         print("--- “)
            result=' '
            visitedstack[]
def main():
    visitedstack=[]
    IDDFS(start, visitedstack, 9)
    print("IDDFS Traversal from ",start," to ",
    goal," is: ") print(result)
main()
```

Output:-

```
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct  2 2023, 13:03:39) [MSC v.1935 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

= RESTART: C:/Users/chika/AIp2.py
Searching at Limit:  0
Not Found!
Arad
-----
Searching at Limit:  1
Not Found!
 Arad Zerind Timisoara Sibiu
-----
Searching at Limit:  2
Not Found!
 Arad Zerind Oradea Timisoara Lugoj Sibiu Rimnicu Fagaras
-----
Searching at Limit:  3
Not Found!
 Arad Zerind Oradea Sibiu Timisoara Lugoj Mehadia
-----
Searching at Limit:  4
Not Found!
 Arad Zerind Oradea Sibiu Rimnicu Fagaras Timisoara Lugoj Mehadia Drobeta
-----
Searching at Limit:  5
Found
IDDFS Traversal from  Arad  to  Bucharest  is:
 Arad Zerind Oradea Sibiu Rimnicu Pitesti Craiova Fagaras Bucharest
```

Practical No: 3

Aim: -Implement the A* Search algorithm for solving pathfinding problem.

Input:-

```
import queue as Q
from RMP import
dict_gn from RMP
import dict_hn


start='Arad'
goal='Bucharest'
result=''


def get_fn(citystr):
   cities=citystr.split(" , ")
   hn=gn=0
   for ctr in range(0, len(cities)-1):
      gn=gn+dict_gn[cities[ctr]][cities[ctr+1]]
   hn=dict_hn[cities[len(cities)-
   1]] return(hn+gn)
def expand(cityq):
   global result
   tot, citystr,
   thiscity=cityq.get() if
   thiscity==goal:
      result=citystr+" : :
      "+str(tot) return
   for cty in dict_gn[thiscity]:
      cityq.put((get_fn(citystr+" , "+cty), citystr+" , "+cty,
```

```
    cty)) expand(cityq)
def main():
    cityq=Q.PriorityQueue()
    thiscity=start
    cityq.put((get_fn(start),start,thiscity))
    expand(cityq)
    print("The A* path with the total is:
    ") print(result)
main()
```

Output:-

```
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct  2 2023, 13:03:39) [MSC v.1935 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

= RESTART: C:/Users/chika/AIp2.py
The A* path with the total is:
Arad , Sibiu , Rimnicu , Pitesti , Bucharest : : 418
```

Practical No: 4

Aim:- Implement the Recursive Best-First Search algorithm for the same problem. Input:-

```
import queue as Q
from RMP import
dict_gn from RMP
import dict_hn
start='Arad'
goal='Bucharest'
result="
def get_fn(citystr):
  cities=citystr.split
  (',') hn=gn=0
  for ctr in range(0,len(cities)-1):
    gn=gn+dict_gn[cities[ctr]][cities[ctr+1]]
  hn=dict_hn[cities[len(cities)-
  1]] return(hn+gn)
def printout(cityq):
  for i in
    range(0,cityq.qsize()):
    print(cityq.queue[i])
def expand(cityq):
  global result
  tot,citystr,thiscity=cityq.g
  et() nexttot=999
  if not cityq.empty():
    nexttot,nextcitystr,nextthiscity=cityq.queue[0
    ]
  if thiscity==goal and
    tot<nexttot:
    result=citystr+'::'+str(tot)
    return
  print("Expanded city ----------",thiscity)
  print("Second best f(n) ----------",nexttot)
  tempq=Q.PriorityQueu
  e() for cty in
  dict_gn[thiscity]:
    tempq.put((get_fn(citystr+','+cty),citystr+','+c
  ty,cty)) for ctr in range(1,3):
    ctrtot,ctrcitystr,ctrthiscity=temp
    q.get() if ctrtot<nexttot:
      cityq.put((ctrtot,ctrcitystr,ctrthiscit
    y)) else:
      cityq.put((ctrtot,citystr,thisc
      ity)) break
  printout(cityq)
  expand(cityq)
def main():
  cityq=Q.PriorityQueue()
```

```
    thiscity=start
    cityq.put((999,"NA","NA
    "))
    cityq.put((get_fn(start),start,thiscity))
    expand(cityq)
    print(result)
main()
```

Output:-

```
========== RESTART: C:/Users/AV/Documents/AIp4.py =
Expanded city------------------------------ Arad
Second best f(n)------------------------------ 999
(393, 'Arad,Sibiu', 'Sibiu')
(999, 'NA', 'NA')
(447, 'Arad,Timisoara', 'Timisoara')
Expanded city------------------------------ Sibiu
Second best f(n)------------------------------ 447
(413, 'Arad,Sibiu,Rimnicu', 'Rimnicu')
(415, 'Arad,Sibiu,Fagaras', 'Fagaras')
(447, 'Arad,Timisoara', 'Timisoara')
(999, 'NA', 'NA')
Expanded city------------------------------ Rimnicu
Second best f(n)------------------------------ 415
(415, 'Arad,Sibiu,Fagaras', 'Fagaras')
(417, 'Arad,Sibiu,Rimnicu', 'Rimnicu')
(447, 'Arad,Timisoara', 'Timisoara')
(999, 'NA', 'NA')
Expanded city------------------------------ Fagaras
Second best f(n)------------------------------ 417
(417, 'Arad,Sibiu,Rimnicu', 'Rimnicu')
(450, 'Arad,Sibiu,Fagaras', 'Fagaras')
(447, 'Arad,Timisoara', 'Timisoara')
(999, 'NA', 'NA')
Expanded city------------------------------ Rimnicu
Second best f(n)------------------------------ 447
(417, 'Arad,Sibiu,Rimnicu,Pitesti', 'Pitesti')
(447, 'Arad,Timisoara', 'Timisoara')
(999, 'NA', 'NA')
(450, 'Arad,Sibiu,Fagaras', 'Fagaras')
(526, 'Arad,Sibiu,Rimnicu', 'Rimnicu')
Expanded city------------------------------ Pitesti
Second best f(n)------------------------------ 447
(418, 'Arad,Sibiu,Rimnicu,Pitesti,Bucharest', 'Bucharest')
(447, 'Arad,Timisoara', 'Timisoara')
(607, 'Arad,Sibiu,Rimnicu,Pitesti', 'Pitesti')
(526, 'Arad,Sibiu,Rimnicu', 'Rimnicu')
(450, 'Arad,Sibiu,Fagaras', 'Fagaras')
(999, 'NA', 'NA')
Arad,Sibiu,Rimnicu,Pitesti,Bucharest::418
```

Practical no.5

Aim:- Implement Decision-Tree learning algorithm.

#REQUIREMENTS:

1)---file1(balance-scale.data) & file2(balance-b

scale.names) 2)---install these modules:

python -mpip install

pandas==0.18 python -m pip

install scipy

python -m pip install

scikit-learn python -m pip

install numpy

Input:-

```
import numpy as
np          import
pandas   as   pd
import    sklearn
as sk
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import
train_test_split from sklearn.tree import
DecisionTreeClassifier from
sklearn.metrics import accuracy_score
from sklearn.metrics import
classification_report #func importing
dataset
def importdata():
  balance_data=pd.read_csv("P5-balance-
 scale.data") #print the dataset shape
  print("Dataset Length : ",len(balance_data))

 #printing the dataset observations
  print("Dataset :
  ",balance_data.head()) return
  balance_data
#func to split the dataset
def splitdataset(balance_data):
 #seperating   the   target
  variable
  X=balance_data.values[:
  ,1:5]
  Y=balance_data.values[
```

```
   :,0]
 #splitting the dataset into train and test
   X_train,X_test,y_train,y_test=train_test_split(X,Y,test_size=0.3,random_
   state=100) return X,Y,X_train,X_test,y_train,y_test
#function to perform training with entropy
def
 train_using_entropy(X_train,X_test,y_train,y_te
 st): #decision tree with entropy

   clf_entropy=DecisionTreeClassifier(criterion="entropy",random_state=100,max_depth=3,min_samp
   les_leaf=
5)
 #performing training
   clf_entropy.fit(X_train,y_
   train) return clf_entropy
def
   prediction(X_test,clf_object):
   y_pred=clf_object.predict(X_t
   est) print("Predicted Values :
   ") print(y_pred)
   return y_pred
def cal_accuracy(y_test,y_pred):
   print("Accuracy : ",accuracy_score(y_test,y_pred)*100)

def main():
   data=importdata(
   )
   X,Y,X_train,X_test,y_train,y_test=splitdataset(data)

   clf_entropy=train_using_entropy(X_train,X_test,y_train
   ,y_test) print("Results using entropy : ")
   y_pred_entropy=prediction(X_test,clf_entropy)
   cal_accuracy(y_test,y_pred_entropy)
main()
```

P-5 Balance -scale- data

B,1,1,1,1
R,1,1,1,2
R,1,1,1,3
R,1,1,1,4
R,1,1,1,5
R,1,1,2,1
R,1,1,2,2
R,1,1,2,3
R,1,1,2,4
R,1,1,2,5
R,1,1,3,1
R,1,1,3,2
R,1,1,3,3
R,1,1,3,4
R,1,1,3,5
R,1,1,4,1
R,1,1,4,2
R,1,1,4,3
R,1,1,4,4
R,1,1,4,5
R,1,1,5,1
R,1,1,5,2
R,1,1,5,3
R,1,1,5,4
R,1,1,5,5
L,1,2,1,1
B,1,2,1,2
R,1,2,1,3
R,1,2,1,4
R,1,2,1,5
B,1,2,2,1
R,1,2,2,2
R,1,2,2,3
R,1,2,2,4
R,1,2,2,5
R,1,2,3,1
R,1,2,3,2
R,1,2,3,3
R,1,2,3,4
R,1,2,3,5
R,1,2,4,1
R,1,2,4,2

R,1,2,4,3
R,1,2,4,4
R,1,2,4,5
R,1,2,5,1
R,1,2,5,2
R,1,2,5,3
R,1,2,5,4
R,1,2,5,5
L,1,3,1,1
L,1,3,1,2
B,1,3,1,3
R,1,3,1,4
R,1,3,1,5
L,1,3,2,1
R,1,3,2,2
R,1,3,2,3
R,1,3,2,4
R,1,3,2,5
B,1,3,3,1
R,1,3,3,2
R,1,3,3,3
R,1,3,3,4
R,1,3,3,5
R,1,3,4,1
R,1,3,4,2
R,1,3,4,3
R,1,3,4,4
R,1,3,4,5
R,1,3,5,1
R,1,3,5,2
R,1,3,5,3
R,1,3,5,4
R,1,3,5,5
L,1,4,1,1
L,1,4,1,2
L,1,4,1,3
B,1,4,1,4
R,1,4,1,5
L,1,4,2,1
B,1,4,2,2
R,1,4,2,3
R,1,4,2,4
R,1,4,2,5
L,1,4,3,1

R,1,4,3,2
R,1,4,3,3
R,1,4,3,4
R,1,4,3,5
B,1,4,4,1
R,1,4,4,2
R,1,4,4,3
R,1,4,4,4
R,1,4,4,5
R,1,4,5,1
R,1,4,5,2
R,1,4,5,3
R,1,4,5,4
R,1,4,5,5
L,1,5,1,1
L,1,5,1,2
L,1,5,1,3
L,1,5,1,4
B,1,5,1,5
L,1,5,2,1
L,1,5,2,2
R,1,5,2,3
R,1,5,2,4
R,1,5,2,5
L,1,5,3,1
R,1,5,3,2
R,1,5,3,3
R,1,5,3,4
R,1,5,3,5
L,1,5,4,1
R,1,5,4,2
R,1,5,4,3
R,1,5,4,4
R,1,5,4,5
B,1,5,5,1
R,1,5,5,2
R,1,5,5,3
R,1,5,5,4
R,1,5,5,5
L,2,1,1,1
B,2,1,1,2
R,2,1,1,3
R,2,1,1,4
R,2,1,1,5

B,2,1,2,1
R,2,1,2,2
R,2,1,2,3
R,2,1,2,4
R,2,1,2,5
R,2,1,3,1
R,2,1,3,2
R,2,1,3,3
R,2,1,3,4
R,2,1,3,5
R,2,1,4,1
R,2,1,4,2
R,2,1,4,3
R,2,1,4,4
R,2,1,4,5
R,2,1,5,1
R,2,1,5,2
R,2,1,5,3
R,2,1,5,4
R,2,1,5,5
L,2,2,1,1
L,2,2,1,2
L,2,2,1,3
B,2,2,1,4
R,2,2,1,5
L,2,2,2,1
B,2,2,2,2
R,2,2,2,3
R,2,2,2,4
R,2,2,2,5
L,2,2,3,1
R,2,2,3,2
R,2,2,3,3
R,2,2,3,4
R,2,2,3,5
B,2,2,4,1
R,2,2,4,2
R,2,2,4,3
R,2,2,4,4
R,2,2,4,5
R,2,2,5,1
R,2,2,5,2
R,2,2,5,3
R,2,2,5,4

R,2,2,5,5
L,2,3,1,1
L,2,3,1,2
L,2,3,1,3
L,2,3,1,4
L,2,3,1,5

Output:-

```
========== RESTART:
Dataset Length :  624
Dataset :     B  1  1.1  1.2  1.3
0  R  1    1    1    2
1  R  1    1    1    3
2  R  1    1    1    4
3  R  1    1    1    5
4  R  1    1    2    1
Results using entropy :
Predicted Values :
['R' 'L' 'R' 'L' 'L' 'L' 'L' 'L' 'R' 'R' 'R' 'L' 'L' 'R' 'R' 'R' 'L' 'L'
 'R' 'R' 'L' 'R' 'R' 'L' 'R' 'R' 'R' 'R' 'R' 'L' 'L' 'L' 'R' 'L' 'R' 'R'
 'R' 'L' 'R' 'L' 'L' 'L' 'R' 'L' 'R' 'R' 'R' 'R' 'R' 'L' 'R' 'R' 'R' 'R'
 'R' 'R' 'R' 'R' 'L' 'L' 'R' 'L' 'R' 'L' 'L' 'R' 'R' 'R' 'R' 'L' 'R' 'L'
 'R' 'L' 'L' 'R' 'L' 'R' 'R' 'R' 'R' 'R' 'L' 'L' 'R' 'R' 'R' 'L' 'R' 'L'
 'R' 'R' 'L' 'L' 'R' 'R' 'R' 'L' 'L' 'R' 'L' 'L' 'L' 'R' 'L' 'R' 'R' 'R'
 'L' 'L' 'R' 'R' 'R' 'R' 'L' 'R' 'R' 'R' 'R' 'R' 'L' 'L' 'R' 'R' 'L' 'L'
 'R' 'L' 'R' 'R' 'L' 'R' 'L' 'L' 'R' 'R' 'R' 'R' 'R' 'R' 'L' 'R' 'R' 'R'
 'R' 'L' 'R' 'L' 'L' 'R' 'R' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'R' 'L'
 'R' 'L' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'L' 'R' 'R' 'R' 'L' 'L' 'R' 'R' 'R'
 'R' 'R' 'R' 'L' 'R' 'L' 'R' 'R']
Accuracy :  66.48936170212765
```

Practical No: 6

Aim:- Implement Naive-Bayes learning algo for RWP(Restaurant Waiting Problem).

Input:-

```
from RWP import
rwp_examples total_exp = 12
def tot(attribute,
  value): count = 0
  for key, val in
    rwp_examples.items(): for
    key1, val1 in val.items():
      if key1 == attribute:
        if val1 == value:
          count += 1
  return count
def getProbab(attribute, attribval,
  value): count = 0
  for key, val in
    rwp_examples.items(): val1 =
    rwp_examples[key][attribute]
    val2 =
    rwp_examples[key]['ans']
    if val1 == attribval and val2 == value:
      count += 1
  probab = count / tot('ans',
  value) return probab
def main():
  PAltYes = tot('Alt', 'Y') /
  total_exp PAltNo = tot('Alt', 'N') /
  total_exp PBarYes = tot('Bar', 'Y')
  / total_exp PBarNo = tot('Bar',
  'N') / total_exp PFriYes = tot('Fri',
  'Y') / total_exp PFriNo = tot('Fri',
  'N') / total_exp PHunYes =
  tot('Hun', 'Y') / total_exp PHunNo
  = tot('Hun', 'N') / total_exp
  PPatSome = tot('Pat', 'S') /
  total_exp PPatFull = tot('Pat', 'F')
  / total_exp PPatNone = tot('Pat',
  'N') / total_exp PPriceCheap =
  tot('Price', '$') / total_exp
  PPriceAvg = tot('Price', '$$') /
  total_exp PPriceExp = tot('Price',
  '$$$') / total_exp PRainYes =
  tot('Rain', 'Y') / total_exp
  PRainNo = tot('Rain', 'N') /
  total_exp PResYes = tot('Res',
```

```
'Y') / total_exp PResNo =
tot('Res', 'N') / total_exp
PTypeFrench = tot('Type', 'F') /
total_exp PTypeItalian =
tot('Type', 'I') / total_exp
PTypeBurger = tot('Type', 'B') /
total_exp PTypeThai = tot('Type',
'T') / total_exp PEstFew =
tot('Est', '0-10') / total_exp
PEstMore = tot('Est', '10-30') /
total_exp
PEstStillMore = tot('Est', '30-60') /
total_exp PEstTooMuch = tot('Est',
'>60') / total_exp PAnsYes =
tot('ans', 'Y') / total_exp PAnsNo =
tot('ans', 'N') / total_exp
print('Probability for will wait if there is an Alternate Restaurant
Nearby: ') print('Yes: Will Wait ', (getProbab('Alt', 'Y', 'Y') *
PAnsYes/PAltYes) * 100, '%') print('No: Will Wait ',
(getProbab('Alt', 'Y', 'N') * PAnsNo/PAltYes ) * 100, '%')
print('Probability for will wait  if there No is an Alternate
Restaurant Nearby: ') print('Yes: Will Wait ', (getProbab('Alt', 'N',
'Y') * PAnsYes/PAltNo) * 100, '%') print('No: Will Wait ',
(getProbab('Alt', 'N', 'N') * PAnsNo/PAltNo) * 100, '%')
print('Probability for will wait if Estimated Wait time is 0-10
minutes: ')
print('Yes: Will Wait ', (getProbab('Est', '0-10', 'Y') * PAnsYes/PEstFew) *
100, '%') print('No: Will Wait ', (getProbab('Est', '0-10', 'N') *
PAnsNo/PEstFew) * 100, '%') print('Probability for will wait if Estimated
Wait time is 10-30 minutes ')
print('Yes: Will Wait ', (getProbab('Est', '10-30', 'Y') *
PAnsYes/PEstMore) * 100, '%') print('No: Will Wait ', (getProbab('Est',
'10-30', 'N') * PAnsNo/PEstMore) * 100, '%') print("Probability for Will
Wait if the Estimated Wait Time is 30-60 mins: ") print("Yes: Will Wait:
",(getProbab('Est','30-60','Y')*PAnsYes/PEstStillMore)*100,"%")
print("No: Will Wait: ",(getProbab('Est','30-
60','N')*PAnsNo/PEstStillMore)*100,"%") print("Probability for Will
Wait if the Estimated Wait Time is >60 mins: ")
print("Yes: Will Wait:
",(getProbab('Est','>60','Y')*PAnsYes/PEstTooMuch)*100,"%") print("No:
Will Wait: ",(getProbab('Est','>60','N')*PAnsNo/PEstTooMuch)*100,"%")
print('Probability for will wait  if there are Some Patrons ')
print('Yes: Will Wait ', (getProbab('Pat', 'S', 'Y') * PAnsYes/PPatSome) *
100, '%') print('No: Will Wait ', (getProbab('Pat', 'S', 'N') *
PAnsNo/PPatSome) * 100, '%') print("Probability for Will Wait if there are
None Patrons: ")
print("Yes: Will Wait:
",(getProbab('Pat','N','Y')*PAnsYes/PPatNone)*100,"%")
print("No: Will Wait:
",(getProbab('Pat','N','N')*PAnsNo/PPatNone)*100,"%")
print("Probability for Will Wait  if there are Full Patrons: ")
```

```
    print("Yes: Will Wait:
    ",(getProbab('Pat','F','Y')*PAnsYes/PPatFull)*100,"%")  print("No:
    Will Wait: ",(getProbab('Pat','F','N')*PAnsNo/PPatFull)*100,"%")
    print('Probability for will wait  if the place is Thai ')
    print('Yes: Will Wait ', (getProbab('Type', 'T', 'Y') * PAnsYes/PTypeThai) *
    100, '%') print('No: Will Wait ', (getProbab('Type', 'T', 'N') *
    PAnsNo/PTypeThai) * 100, '%')
main()
```

Output:-

```
========== RESTART: C:\Users\AV\Documents\P6.py ==========
Probability for will wait if there is an Alternate Restaurant Nearby:
Yes: Will Wait  50.0 %
No: Will Wait  50.0 %
Probability for will wait if there No is an Alternate Restaurant Nearby:
Yes: Will Wait  50.0 %
No: Will Wait  50.0 %
Probability for will wait if Estimated Wait time is 0-10 minutes:
Yes: Will Wait  71.42857142857143 %
No: Will Wait  28.57142857142857 %
Probability for will wait if Estimated Wait time is 10-30 minutes
Yes: Will Wait  50.0 %
No: Will Wait  50.0 %
Probability for Will Wait if the Estimated Wait Time is 30-60 mins:
Yes: Will Wait:  0.0 %
No: Will Wait:  100.0 %
Probability for Will Wait if the Estimated Wait Time is >60 mins:
Yes: Will Wait:  0.0 %
No: Will Wait:  100.0 %
Probability for will wait if there are Some Patrons
Yes: Will Wait  100.0 %
No: Will Wait  0.0 %
Probability for Will Wait if there are None Patrons:
Yes: Will Wait:  0.0 %
No: Will Wait:  100.0 %
Probability for Will Wait if there are Full Patrons:
Yes: Will Wait:  33.33333333333333 %
No: Will Wait:  66.66666666666666 %
Probability for will wait if the place is Thai
Yes: Will Wait  50.0 %
No: Will Wait  50.0 %
```

Practical No: 7

Aim:- Implement feed forward back propagation neural network

learning algorithm. Input:-

```
import numpy as np

class
  NeuralNetwork()
  : def _init_
  (self):
    #seeding for random number
    generation np.random.seed()

    #converting weights to a 3 by 1 matrix
    self.synaptic_weights=2*np.random.random
    ((3,1))-1

  #x is output
  variable def
  sigmoid(self, x):
    #applying the sigmoid
    function return
    1/(1+np.exp(-x))

  def sigmoid_derivative(self,x):
    #computing derivative to the sigmoid
    function return x*(1-x)

  def train(self,training_inputs,training_outputs,training_iterations):

    #training the model to make accurate predictions while
    adjusting for iteration in range(training_iterations):
      #siphon the training data via the neuron
      output=self.think(training_inputs)

      error=training_outputs-output

      #performing weight adjustments
      adjustments=np.dot(training_inputs.T,error*self.sigmoid_derivative(output))

      self.synaptic_weights+=adjustments

  def think(self,inputs):
    #passing the inputs via the neuron to get
    output #converting values to floats

    inputs=inputs.astype(float)
    output=self.sigmoid(np.dot(inputs,self.synaptic_weigh
    ts))

    return output
```

```
if__name_=="__main__":

   #initializing the neuron class
   neural_network=NeuralNetwork()

   print("Beginning randomly generated
   weights: ")
   print(neural_network.synaptic_weights)

   #training data consisting of 4 examples--3 inputs & 1
   output
   training_inputs=np.array([[0,0,1],[1,1,1],[1,0,1],[0,1,1]]
   ) training_outputs=np.array([[0,1,1,0]]).T


   #training taking place
   neural_network.train(training_inputs,training_outputs,15
   000)

   print("Ending weights after training: ")
   print(neural_network.synaptic_weights)

   user_input_one=str(input("User Input One:
   ")) user_input_two=str(input("User Input
   Two: ")) user_input_three=str(input("User
   Input Three: "))

   print("Considering new situation:
   ",user_input_one,user_input_two,user_input_three) print("New output data: ")
   print(neural_network.think(np.array([user_input_one,user_input_two,user_input_th
   ree])))
```

Output:-

```
================== RESTART: C:\Users\AV\Docu
Beginning randomly generated weights:
[[ 0.67813245]
 [ 0.90926853]
 [-0.4150907 ]]
Ending weights after training:
[[10.08758208]
 [-0.206867  ]
 [-4.83771668]]
User Input One: 2
User Input Two: 4
User Input Three: 6
Considering new situation:  2 4 6
New output data:
[6.26049311e-05]
>>>
```

Practical No: 8

Aim:- Implement AdaBoost(Adaptive Boosting)

learning algorithm Input:-

```
import pandas
from sklearn import model_selection
from sklearn.ensemble import AdaBoostClassifier
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-
diabetes.data.csv" names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age',
'class']
dataframe = pandas.read_csv(url,
names=names) array = dataframe.values
X = array[:,0:8]
Y = array[:,8]
seed = 7
num_trees = 30

model = AdaBoostClassifier(n_estimators=num_trees, random_state=seed)

results =
model_selection.cross_val_score(model, X, Y)
print(results.mean())
```

Output:-

```
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct  2 2023, 13:03:39) [MSC v.1935 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

========== RESTART: C:/Users/chika/AIp6.py ==========
0.7617774382480265
```

Practical No: 9

Aim:- Implement Reinforcement Learning

algorithm.

#REQUIREMENT: (T.npy) file

T.npy File

```
[[[0.9, 0.9, 0.1, 0.1],
  [0.1, 0, 0.1, 0.8],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0.1, 0.8, 0.1],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0]],
 [[0.1, 0.8, 0.1, 0],
  [0.8, 0.2, 0.8, 0.2],
  [0.1, 0, 0.1, 0.8],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0]],
 [[0, 0, 0, 0],
  [0.1, 0.8, 0.1, 0],
  [0.8, 0.1, 0, 0.1],
  [0.1, 0, 0.1, 0.8],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0.1, 0.8, 0.1],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
```

```
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0]],
 [[0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0]],
 [[0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0.8, 0.1, 0, 0.1],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0.1, 0.9, 0.9, 0.1],
  [0.1, 0, 0.1, 0.8],
  [0, 0, 0, 0],
  [0, 0, 0, 0]],
 [[0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0.1, 0.8, 0.1, 0],
  [0.8, 0.2, 0.8, 0.2],
  [0.1, 0, 0.1, 0.8],
  [0, 0, 0, 0]],
 [[0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
```

```
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0.8, 0.1, 0, 0.1],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0.1, 0.8, 0.1, 0],
  [0, 0.1, 0.8, 0.1],
  [0.1, 0, 0.1, 0.8]],
 [[0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0.8, 0.1, 0, 0.1],
  [0, 0, 0, 0],
  [0, 0, 0, 0],
  [0.1, 0.8, 0.1, 0],
  [0.1, 0.1, 0.9, 0.9]]]
```

Input:-

```
import numpy as np
def return_state_utility(v, T, u, reward, gamma):
action_array = np.zeros(4)
for action in range(0, 4):
action_array[action] = np.sum(np.multiply(u, np.dot(v, T[:,:,action]))) return reward + gamma *
np.max(action_array)
def main():
v = np.array([[0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0,
1.0, 0.0, 0.0, 0.0]])
T= np.load("T.npy")
u = np.array([[0.812, 0.868, 0.918, 1.0, 0.762, 0.0, 0.660, -1.0,
0.705, 0.655, 0.611, 0.388]])
reward = -0.4
gamma
1.0
utility_11 = return_state_utility(v, T, u, reward, gamma)
print("Utility of state (1,1): " + str(utility_11))main()
```

Output:

